Dental Booking Platform

Multi-Tenant Architecture & Development Plan

Document type: Technical stakeholder architecture

Date: May 2026 · Status: Planning

Audience: Technical leads, architects, engineering stakeholders

Confidential

Table of Contents

1. Executive Summary

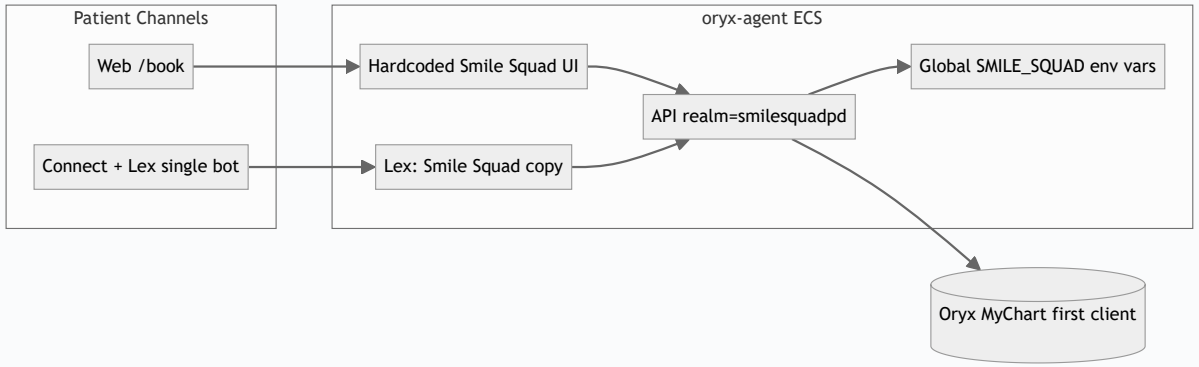
The **Dental Booking Platform** evolves the existing single-practice *oryx-agent* application (currently deployed for Smile Squad Pediatric Dentistry) into a **multi-tenant SaaS product** sold on a **monthly subscription** basis.

> **Core value proposition:** Dental clinics get one or more **Amazon Connect + Lex voice bots** (e.g. appointment booking, administrative tasks) that integrate with their Practice Management System (PMS). An optional **Web Booking Form** can be enabled as a separate subscription feature. **AWS only** for voice and platform services in the current scope (no third-party CRM/voice middleware).

Decision	Choice
Cloud scope	**Amazon AWS only** — Connect, Lex V2, ECS, RDS, S3, Secrets Manager, Stripe billing
Deployment model	Single shared ECS deployment; tenants isolated at app + DB layer
PMS strategy	**PMS-agnostic platform** via adapter layer; target broad dental PMS coverage. **Oryx is the first production client**, not the long-term architectural center
Voice bots per tenant	Up to **8 Lex bots per tenant** (e.g. booking on call, admin, reminders); each bot has its own intents, prompts, and optional DID
Web routing	Path slug `/book/{slug}` (e.g. `/book/smilesquad`) in v1
Voice routing	Amazon Connect + Lex; called DID maps to **tenant + bot**
Onboarding v1	Operator-configured tenants via admin portal
Billing	Stripe monthly subscriptions in Phase 4
First client	Smile Squad on Oryx migrates as tenant #1 (`slug: smilesquad`) with booking bot
LLM (optional)	**Amazon Bedrock** for speech-to-text correction (names, email, corrections) — not required for multi-tenant v1; booking APIs stay deterministic

2. Current State vs Target State

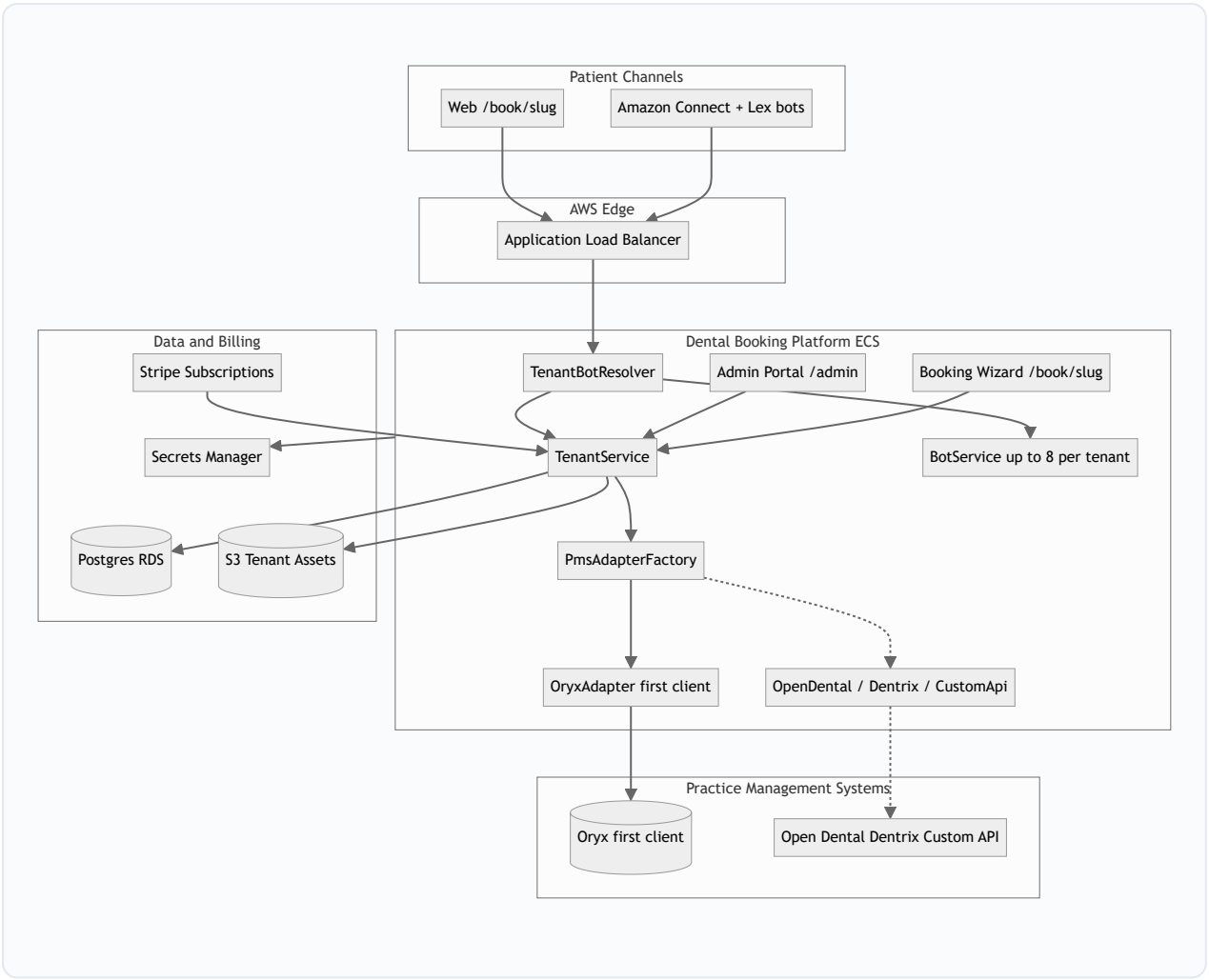
2.1 Today — Single Tenant (Smile Squad)



Current limitations:

- Practice branding, address, phone, and doctor content hardcoded in React components
- Oryx realm `smilesquadpd` hardcoded in all API routes
- Operatory routing rules in global env vars (`SMILE_SQUAD_*`)
- Lex Lambda has ~15 hardcoded Smile Squad voice prompts; duplicates operatory logic
- No tenant registry, database, billing, or admin onboarding

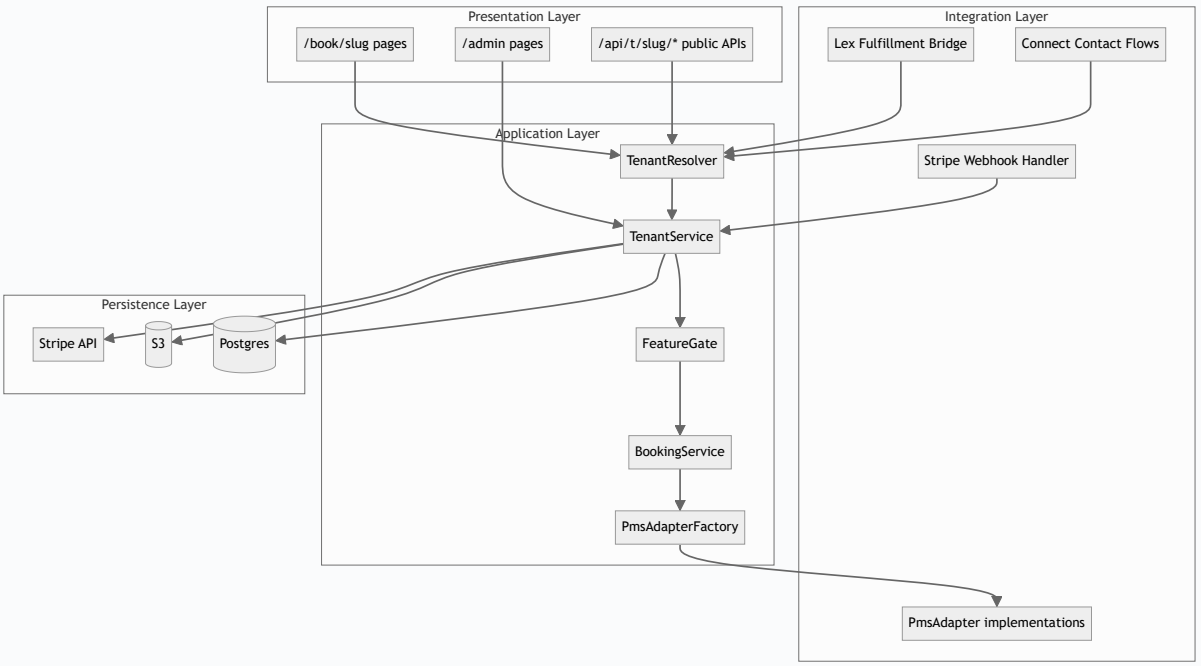
2.2 Target — Multi-Tenant Platform



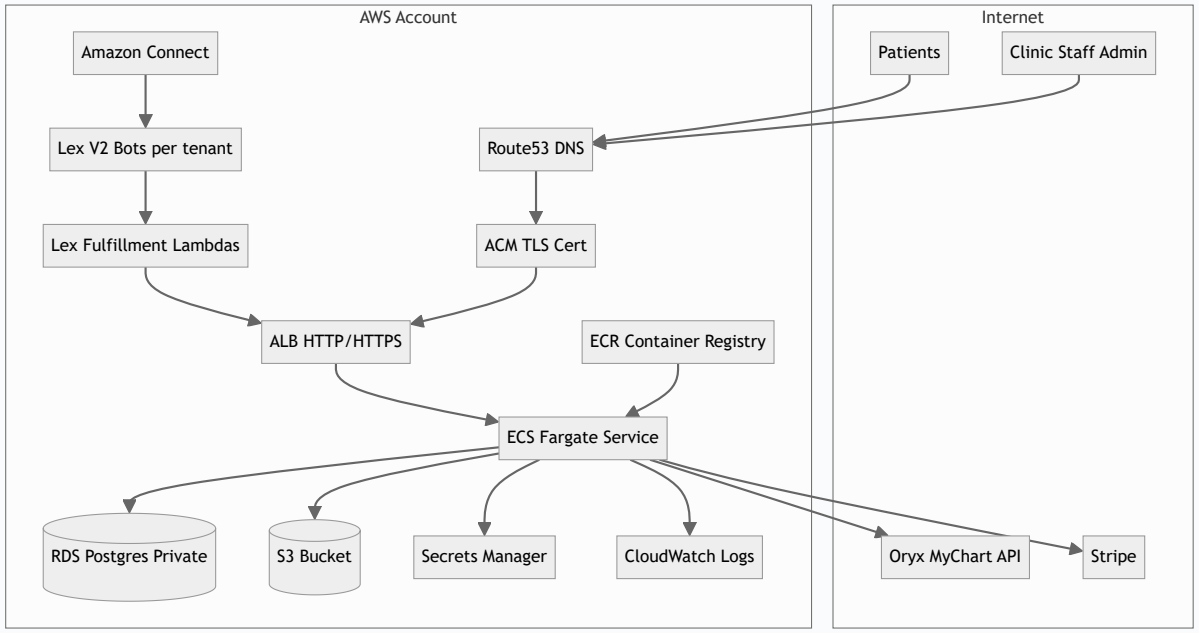
3. System Architecture Overview

The platform is a **Next.js application** on **AWS ECS Fargate** behind an ALB. All booking channels converge on **TenantResolver**, which loads clinic config from **Postgres** and delegates to **PmsAdapter**.

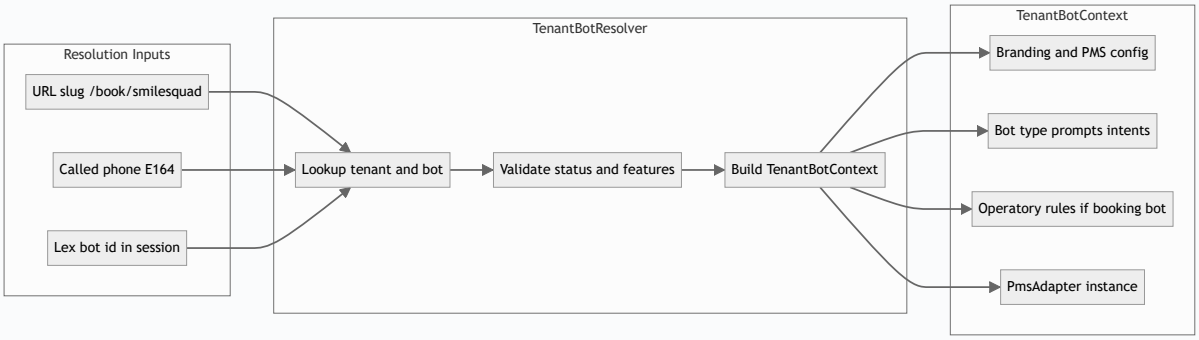
3.1 Logical Component Layers



3.2 AWS Deployment Topology



4. Tenant Resolution & Multi-Tenancy

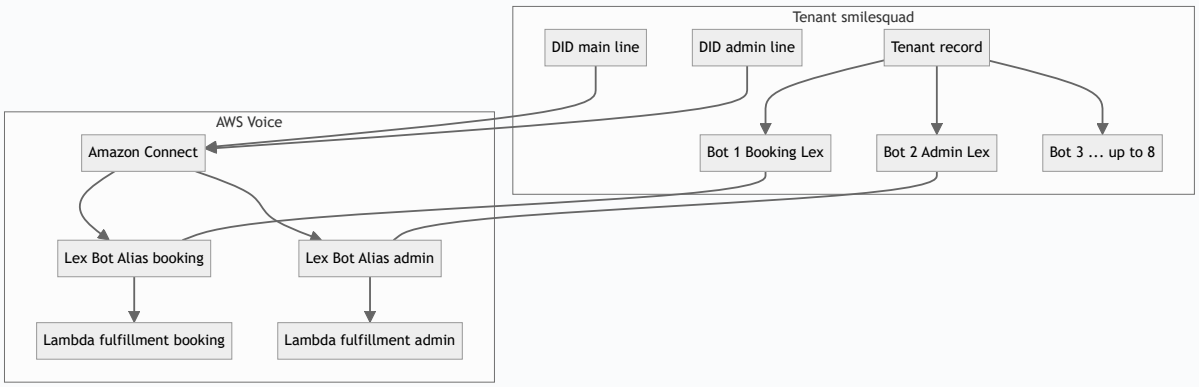


Channel	Resolution key	Behavior
Web booking UI	`/book/[slug]`	404 if inactive or webForm disabled
Web booking API	`/api/t/[slug]/...`	Per-tenant access code, rate limit
Voice (Lex)	Called DID from Connect	`GET /api/internal/resolve-phone` → tenant + bot config
Lex fulfillment	`botId` + tenant slug in session	Routes to correct Lambda handler and prompt set
Internal PMS book	`X-Tenant-Id` + bot capability	Authenticated with platform secret; booking bots only call PMS book APIs

5. Multi-Bot Model per Tenant

A single dental client (tenant) may operate **multiple Amazon Lex V2 bots** — up to **8 per tenant** — each dedicated to a different workflow. Examples:

- **Booking bot** — inbound calls to schedule appointments (current PatientBooking flow)
- **Administrative bot** — office hours, directions, insurance FAQs, transfer to staff
- **Future bots** — reminders, cancellations, new-patient intake, Spanish line, etc.



Concept	Description
`tenant_bots`	Bot registry: `bot_type`, Lex bot ID/alias, Lambda ARN, Connect flow ARN, enabled capabilities
`tenant_phone_numbers`	Maps E164 DID → `tenant_id` + `bot_id` (not tenant alone)
Prompts & intents	Per-bot configuration; booking bot loads operatory rules; admin bot may skip PMS book APIs
Subscription	Plans may cap enabled bot count (e.g. 1, 3, or 8 bots)

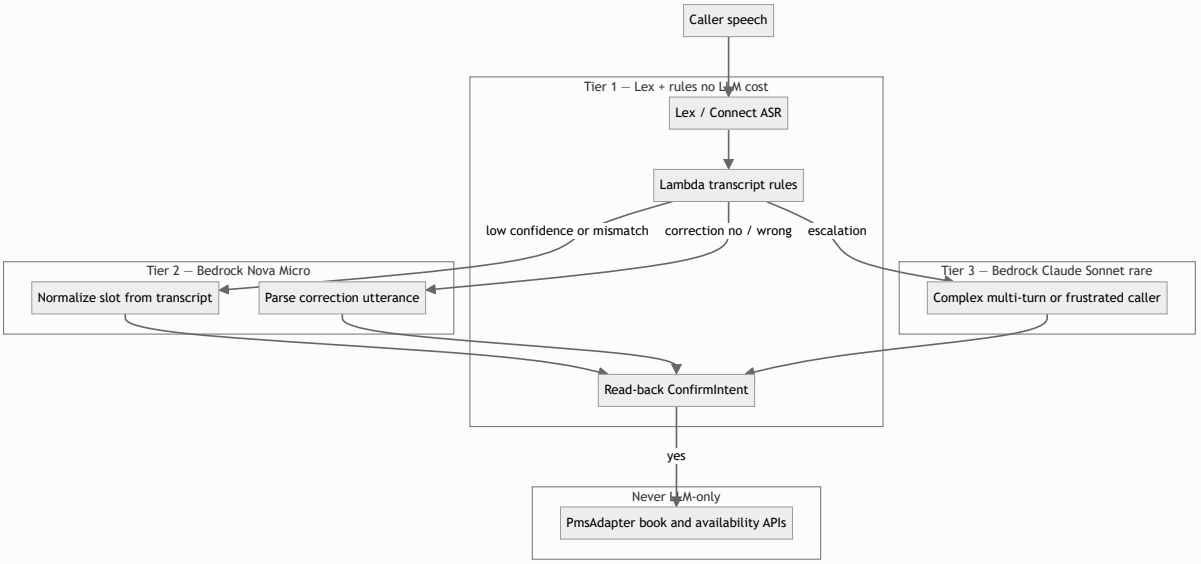
6. LLM & Speech Understanding (ASR Correction)

6.1 Today vs target

Production voice today uses **Amazon Lex V2 + fulfillment Lambda** — not a general-purpose LLM. Lex provides intents, slots, and speech recognition; the Lambda already prefers `inputTranscript` over Lex slot values because **slots are often wrong for name, email, and phone**. Heuristics (spelled-letter names, phone word parsing) help but do not fix all ASR errors or caller corrections.

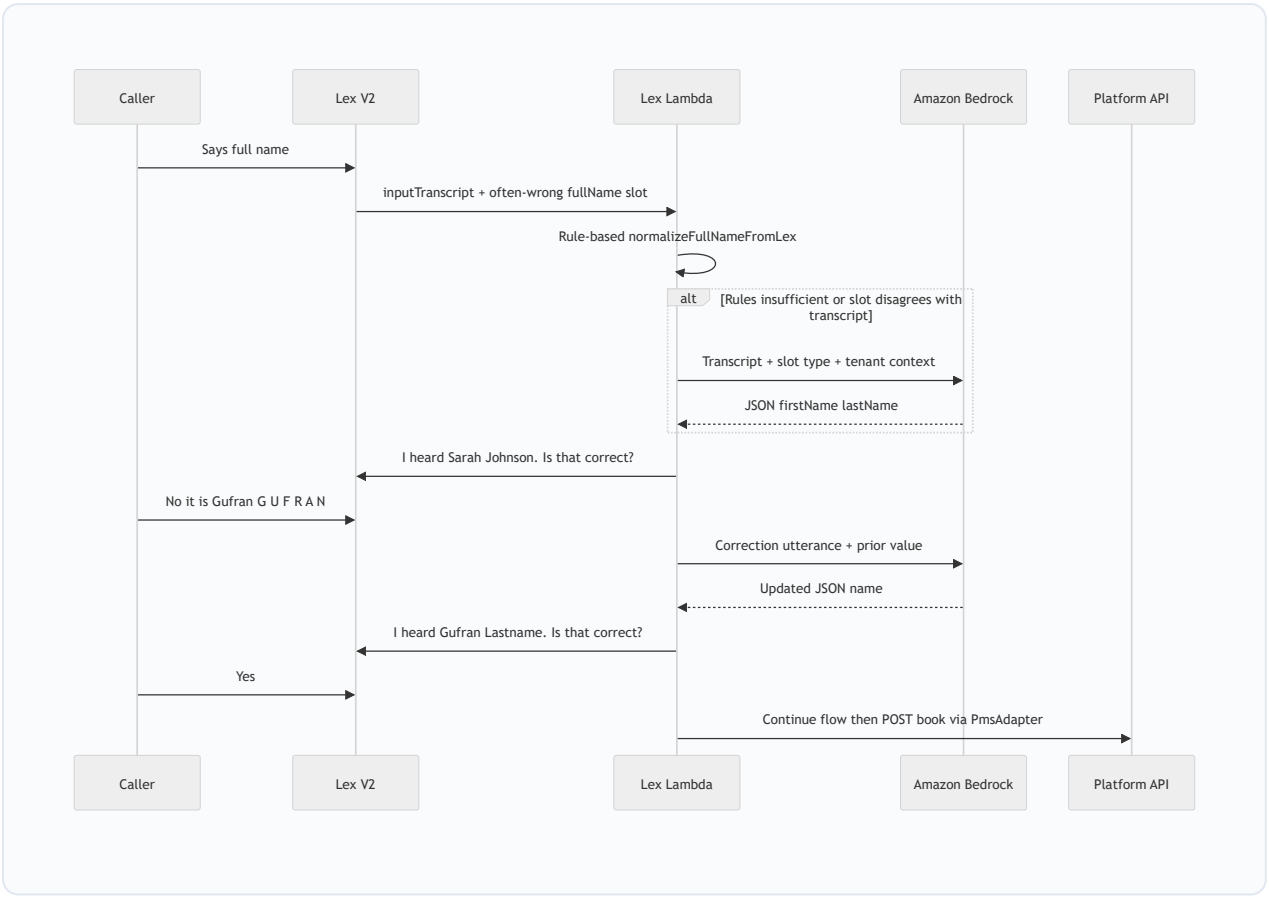
> **Observed pain point:** The bot sometimes captures the **wrong name** or **misheard words** (uncommon names, accents, noise). Target: add an optional **Amazon Bedrock** layer to normalize speech into structured fields and handle "no, I said …" correction turns — while keeping **read-back confirmation** and **deterministic PMS booking APIs**.

6.2 Tiered intelligence model (AWS only)



Tier	Technology	Use when
1	Lex slots + Lambda rules	Happy path; phone DTMF; spelled-letter names
2	Bedrock Nova Micro	Name/email normalization from transcript; simple fallback when Lex does not understand
3	Bedrock Claude Sonnet	Target <5% of calls — complex phrasing, many failed turns, admin FAQ bot

6.3 ASR correction flow (booking bot — name example)



6.4 What the LLM does and does not do

LLM should	LLM should not
Fix likely ASR errors for **name**, **email**, light date phrasing	Choose operatory, time slot, or PMS realm by itself
Understand correction intent (“that’s wrong”, spelled letters)	Skip read-back confirmation before book
Power **admin/FAQ bots** from tenant knowledge	Send unnecessary PHI in prompts (minimize under AWS BAA)
Use per-tenant prompt + optional name vocabulary (doctors, locale)	Replace Lex entirely for structured booking

6.5 Non-LLM mitigations (use alongside Bedrock)

- Always **read back** critical fields and require explicit yes
- Fallback: **spell-by-letter** for names when validation fails
- Tune Connect/Lex speech capture (silence timeouts, VAD)
- **Custom vocabulary** for practice and provider names

- **DTMF** for phone entry where speech fails

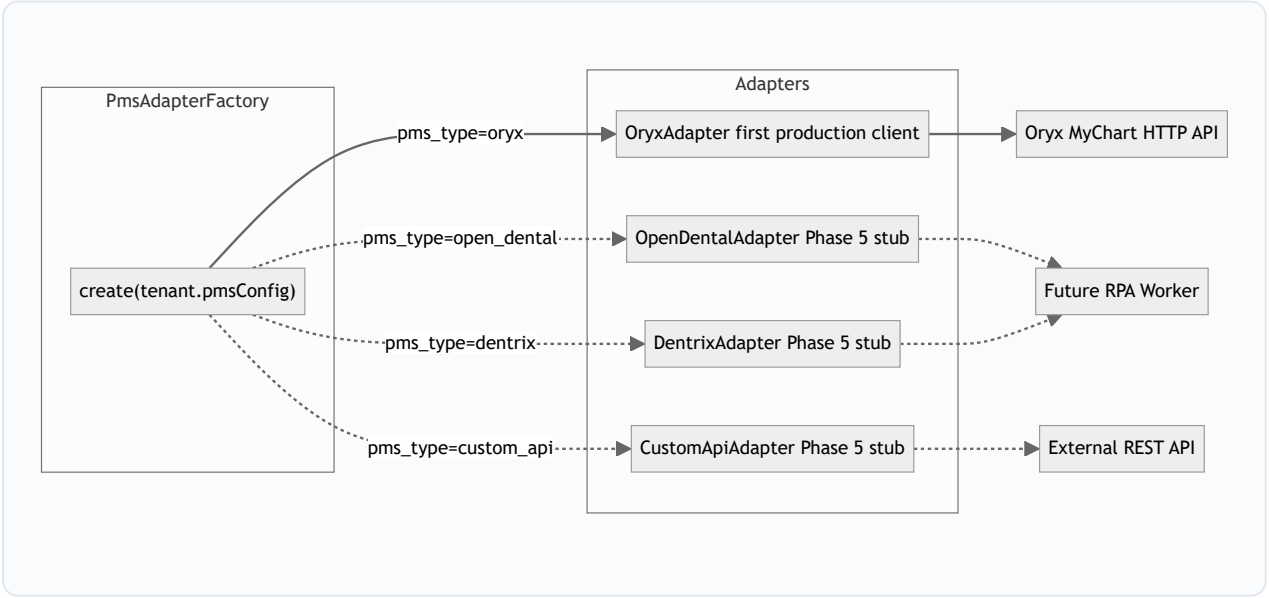
6.6 Platform integration

- **Invoke from Lex Lambda** or `POST /api/internal/normalize-slot` on ECS (tenant + bot + slot type + transcript)
- Tenant flag: `features.llmSlotCorrection` (optional per plan)
- Store `bedrock_model_tier` and prompt templates in `tenant_bots.prompts_config`
- HIPAA: Bedrock under AWS BAA; de-identified prompts; audit logs without full transcript retention where possible

7. PMS Adapter Layer

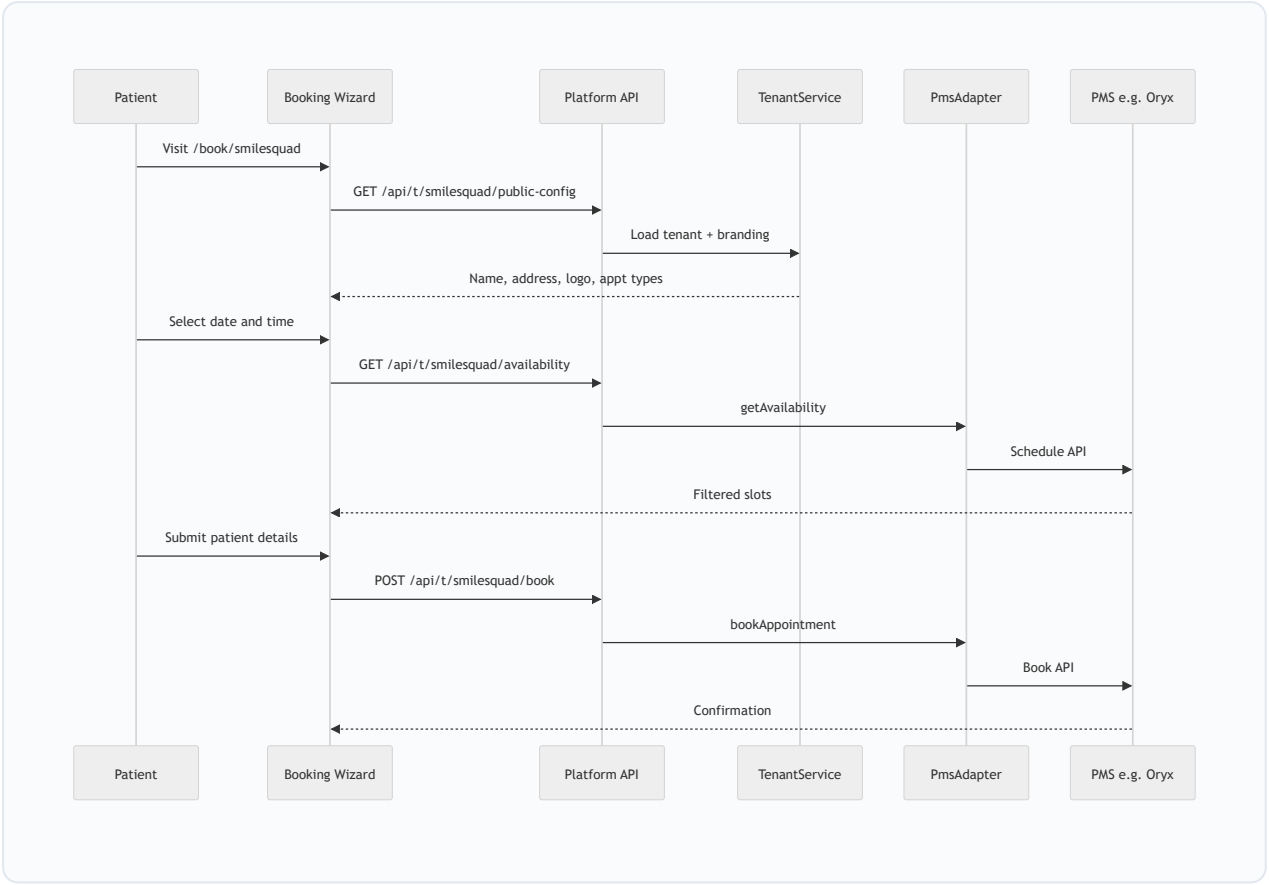
The platform is built **PMS-agnostic**: scheduling and booking semantics are expressed through a stable `PmsAdapter` interface. Implementations target the breadth of dental PMS ecosystems (Open Dental, Dentrix, Eaglesoft, custom APIs, RPA where needed). **Oryx is the first live client adapter** (Smile Squad production), proving the model — it is not a statement that the product is "Oryx-first" long term.

```
interface PmsAdapter {
  getPracticeInfo(): Promise<PracticeInfo>;
  getProviders(): Promise<Provider[]>;
  getAvailability(input: AvailabilityQuery): Promise<Slot[]>;
  bookAppointment(input: BookInput): Promise<BookResult>;
}
```

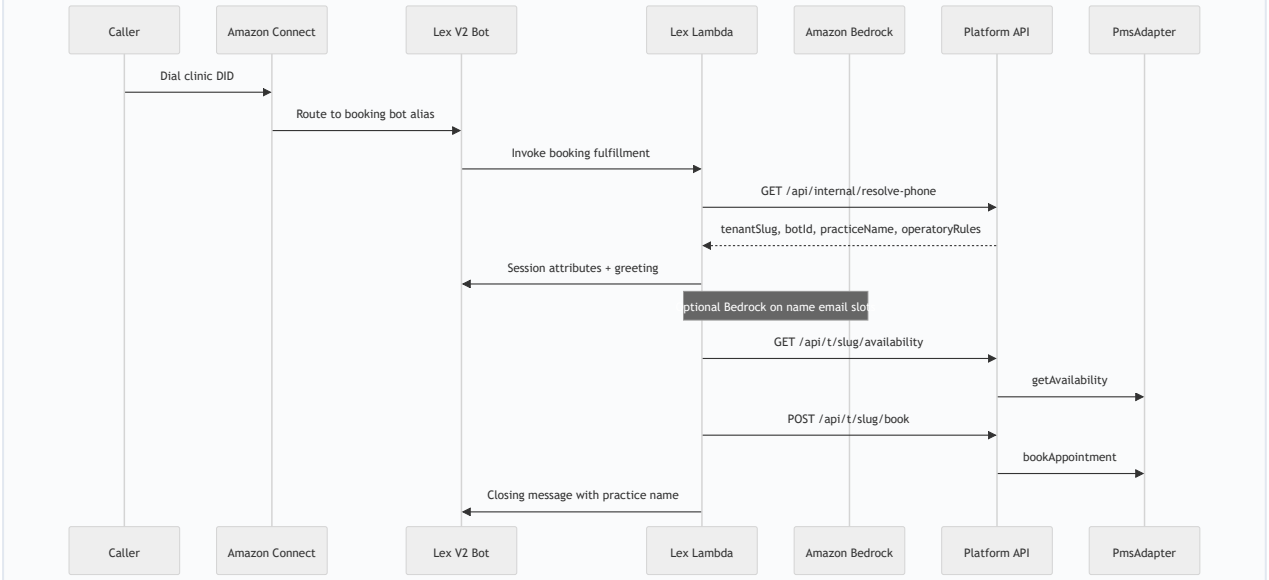


8. Channel Flows

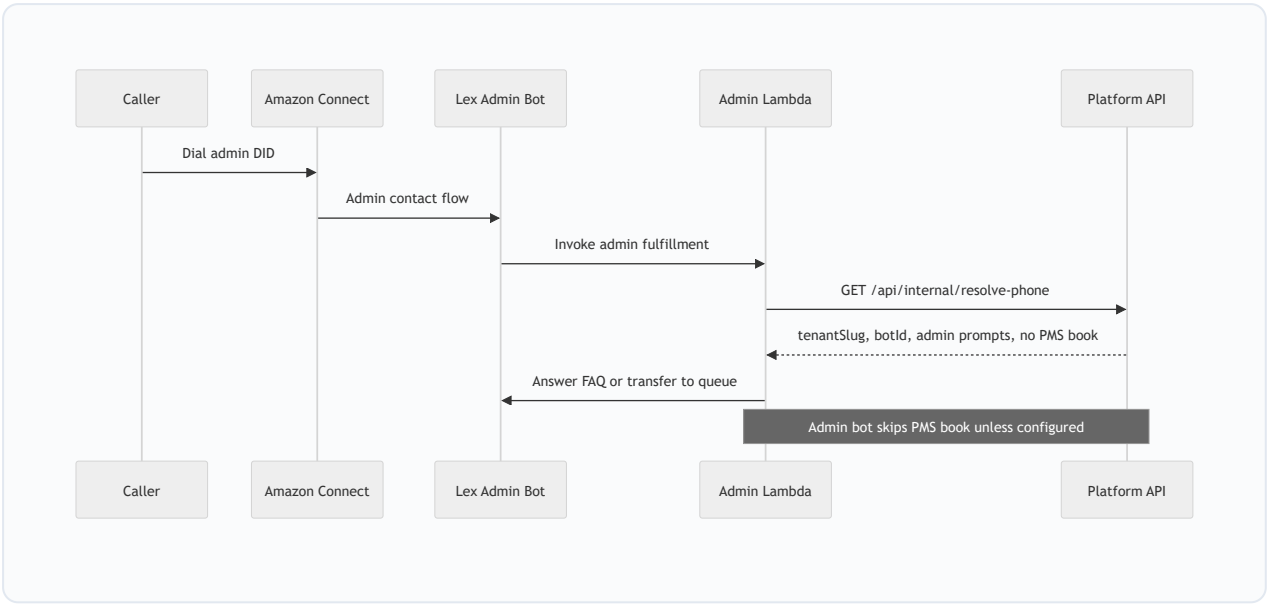
8.1 Web Booking Flow



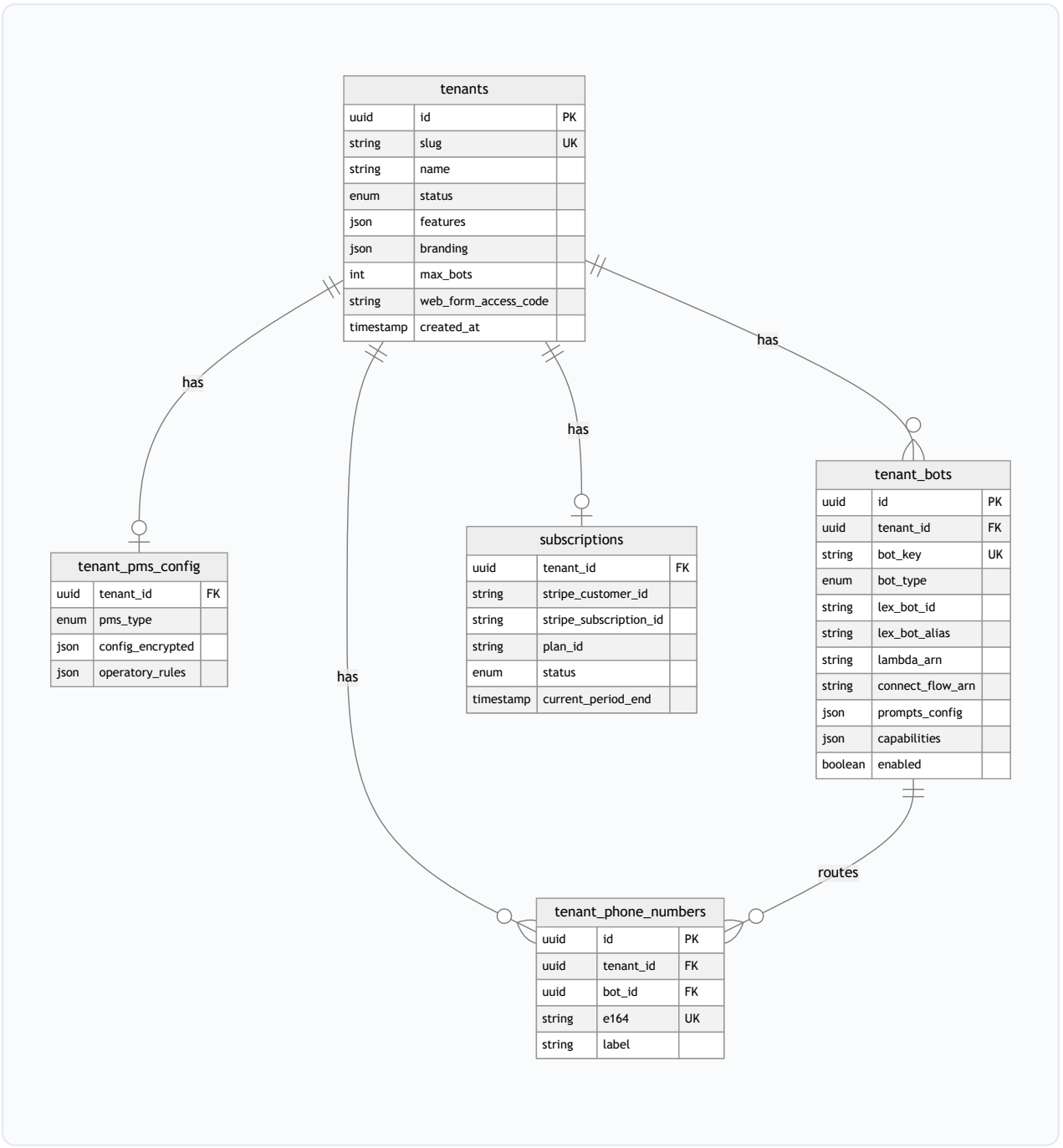
8.2 Voice Agent Flow — Booking Bot



8.3 Voice Agent Flow — Non-Booking Bot (e.g. Admin)



9. Data Model


tenants
uuid id PK
string slug UK
string name
enum status
json features
json branding
int max_bots
string web_form_access_code
timestamp created_at
has
has
has
has
has
tenant_pms_config
uuid tenant_id FK
enum pms_type
json config_encrypted
json operatory_rules
subscriptions
uuid tenant_id FK
string stripe_customer_id
string stripe_subscription_id
string plan_id
enum status
timestamp current_period_end
tenant_bots
uuid id PK
uuid tenant_id FK
string bot_key UK
enum bot_type
string lex_bot_id
string lex_bot_alias
string lambda_arn
string connect_flow_arn
json prompts_config
json capabilities
boolean enabled
routes
tenant_phone_numbers
uuid id PK
uuid tenant_id FK
uuid bot_id FK
string e164 UK
string label

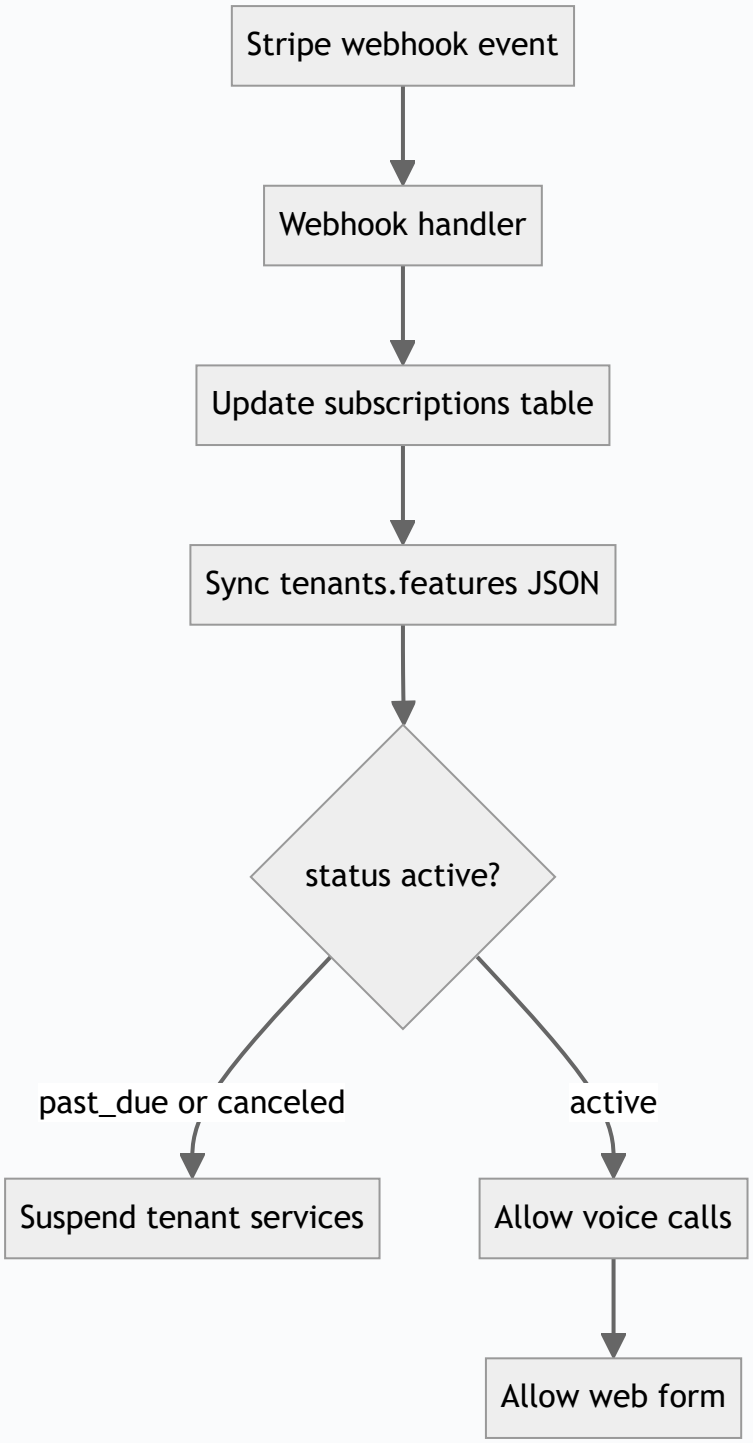
10. API Surface

Endpoint	Method	Auth	Purpose
`/api/t/[slug]/public-config`	GET	None	Branding, appt types, access-code flag
`/api/t/[slug]/availability`	GET	Per-tenant access code	Filtered schedule slots (booking)
`/api/t/[slug]/book`	POST	Access code + rate limit	Book into PMS via adapter
`/api/internal/resolve-phone`	GET	Platform secret	DID → tenant + bot + prompts/capabilities
`/api/internal/normalize-slot`	POST	Platform secret	Transcript → structured slot (Bedrock); name, email, correction
`/api/webhooks/stripe`	POST	Stripe signature	Subscription lifecycle

Legacy GHL routes (`/api/ghl/*`) are out of scope for the AWS-only product roadmap and will not be extended for multi-tenant.

11. Subscription & Feature Gating

Plan	Lex bots	Web Booking Form	Billing
Voice — 1 bot	1 bot (e.g. booking)	No	Stripe monthly
Voice — multi-bot	Up to 8 bots	Optional add-on	Stripe monthly
Voice + Web Form	Per plan tier	Yes	Stripe monthly

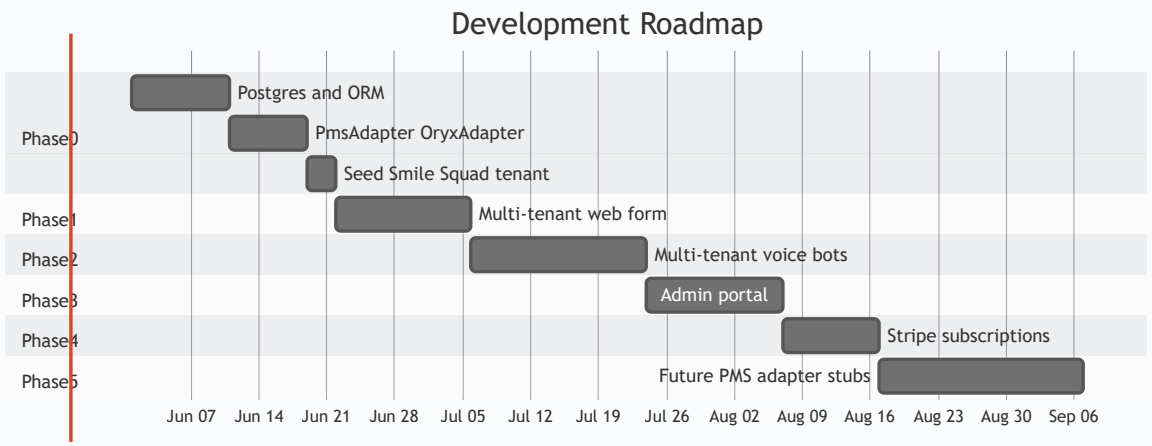

Stripe webhook event
Webhook handler
Update subscriptions table
Sync tenants.features JSON
status active?
past_due or canceled
active
Suspend tenant services
Allow voice calls
Allow web form

12. AWS Infrastructure

Component	Current	Target
Compute	ECS Fargate single service	Same — app-level multi-tenancy
Database	None	RDS Postgres encrypted, private subnet
Secrets	`APP_ENV_JSON`	Add `DATABASE_URL`
Storage	None	S3 for tenant logos and hero images
DNS / TLS	ALB HTTP only	ACM + Route53 wildcard cert
Voice	Connect + Lex + Lambda	Multiple Lex bots per tenant; DID → bot mapping in DB
LLM	None	Amazon Bedrock (Nova / Sonnet) optional for ASR correction and admin bots
Monitoring	CloudWatch logs	Per-tenant booking metrics (Phase 3+)

13. Development Phases & Timeline

Estimated total: 10–14 weeks to full multi-tenant platform with billing.



Phase 0 — Foundation (2–3 weeks)

Prerequisite for all other phases

- RDS Postgres + ORM + migrations
- Tenant schema, `tenant_bots`, TenantService, BotService
- PmsAdapter interface + Oryx adapter (first client)
- Seed Smile Squad with booking bot
- Internal `resolve-phone` API

Phase 1 — Multi-Tenant Web Form (2 weeks)

- `/book/[slug]` with dynamic branding
- Tenant-scoped availability, book, public-config APIs
- Feature gate: 403 if webForm disabled

Phase 2 — Multi-Tenant Voice Bots (2–3 weeks)

- DID → tenant + bot resolution
- Booking bot: templated prompts; secure tenant-aware book API
- Bot registry supports multiple Lex bots per tenant
- Remove duplicated operatory logic from Lambda

Phase 2b — Bedrock ASR correction (1–2 weeks, optional)

- Bedrock Nova: name/email normalization from `inputTranscript`
- Correction-turn handler when caller says name is wrong
- Per-tenant feature flag; keep ConfirmIntent read-back
- Admin bot: heavier Bedrock use for FAQs

Phase 3 — Admin Portal (2 weeks)

- Tenant CRUD, bot configuration (up to 8), phone-to-bot mapping
- S3 asset upload; booking health dashboard

Phase 4 — Stripe Subscriptions (1–2 weeks)

- Products/prices, webhook handler, feature sync, tenant suspension

Phase 5 — PMS Extensibility (ongoing)

- Open Dental, Dentrix, Custom API adapter stubs
- RPA worker path for PMS without public APIs

14. Risks & Success Criteria

Risk	Mitigation
Oryx cookie init latency	Cache OryxClient session per tenant+realm
Lex cold start + tenant/bot lookup	One `resolve-phone` call returns tenant + bot config; cache in session
Many Lex bots per tenant	Bot registry in DB; shared Lambda with bot routing or per-bot Lambdas
LLM hallucinated name	Structured JSON + validation + mandatory read-back before book
Bedrock latency every turn	Invoke only on mismatch, fallback, or correction — not every slot
Breaking Smile Squad production	Parallel routes, slug redirect, per-tenant flags
Operatory ID varies per realm	Per-tenant operatory_rules JSON
PHI in database	Config only in DB; booking payloads ephemeral; encrypt RDS
Single ECS blast radius	Health checks, auto-scaling, CloudWatch alarms

Success Criteria

- Second clinic onboarded via admin with only DB config (no redeploy)
- `/book/{slug}` shows correct branding and books into correct Oryx realm
- Voice call to Clinic B's booking DID uses Clinic B booking bot prompts and PMS config
- Clinic B can enable a second bot (e.g. admin) on a separate DID without redeploy
- Voice-only tenant gets 403 on web form
- PmsAdapterFactory registers a second PMS adapter without UI/Lex booking flow changes
- With Bedrock enabled: uncommon names corrected after read-back at higher rate than rules-only baseline
- Stripe cancellation suspends tenant within one webhook cycle

> **Migration note:** Smile Squad remains operational throughout Phases 0–1 via backward-compatible route wrappers and redirect from `/book` to `/book/smilesquad`.